Joseph Leone
CEN-3062C
Assignment 3
7-29-2026

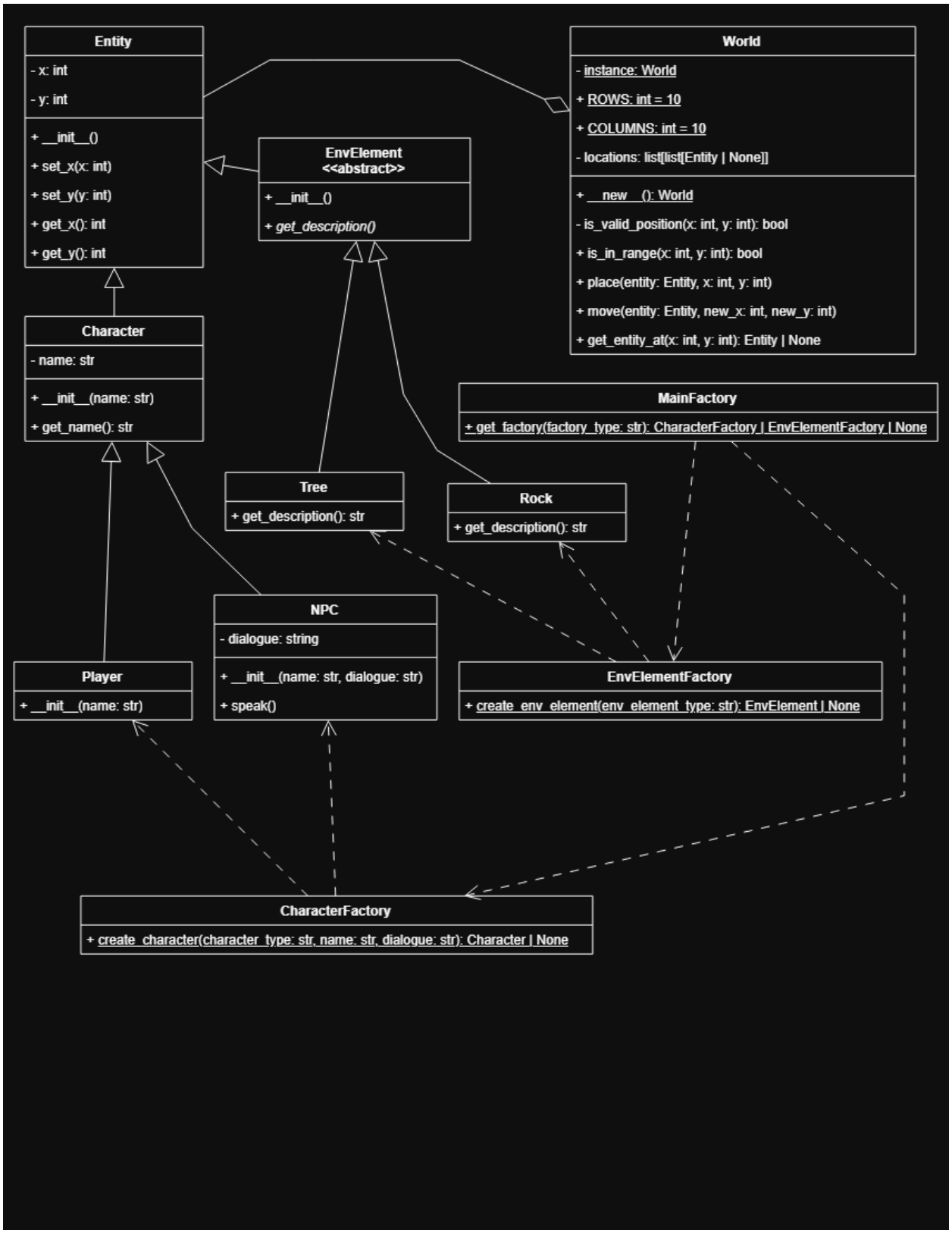
In my program I used the Factory Pattern, Abstract Factory Pattern, and the Singleton Pattern:

1. Factory Pattern
    - The factory pattern allows for a uniform process when creating similar types of objects.
    - The basic idea behind the pattern is that related objects can be created by calling a static method of the factory Class.
    - I use two factories in my program where each can create two types of related objects.
    - The main advantage of this pattern is uniformity and the ability to add new types of objects without having to change the way objects are created.
    - The main disadvantage is that there are more classes required and more code complexity.

2. Abstract Factory Pattern
    - The abstract factory pattern allows for a standard process for creating factories.
    - The basic idea is that when a factory is needed there is a class that handles the creation of those factories.
    - I use one abstract factory in my program that can create the two factories that I use.
    - The main advantage of this pattern is the uniformity of creating a factory. Also the organization aspect of knowing what to use to create any factory.
    - The main disadvantage is added complexity.

3. Singleton Pattern
    - The singleton pattern solves the problem of when there should only be one instance of a class in a program.
    - The basic idea is that the singleton class blocks creation of a second instance of the class and always will return a reference to the first instance of the class.
    - I use the singleton pattern for the World class in my program since there should only be one of them.
    - The main advantage of the singleton class is the security in knowing that whenever there is interaction with an instance of the class that it is the same instance.
    - The main disadvantage is inflexibility by introducing a global state and it is more difficult to make changes to the program in the future.

UML Diagram


Entity
- x: int
- y: int
+ __init__()
+ set_x(x: int)
+ set_y(y: int)
+ get_x(): int
+ get_y(): int
World
- instance: World
+ ROWS: int = 10
+ COLUMNS: int = 10
- locations: list[list[Entity | None]]
+ __new__(): World
- is_valid_position(x: int, y: int): bool
+ is_in_range(x: int, y: int): bool
+ place(entity: Entity, x: int, y: int)
+ move(entity: Entity, new_x: int, new_y: int)
+ get_entity_at(x: int, y: int): Entity | None
EnvElement
<<abstract>>
+ __init__()
+ get_description()
Character
- name: str
+ __init__(name: str)
+ get_name(): str
MainFactory
+ get_factory(factory_type: str): CharacterFactory | EnvElementFactory | None
Tree
+ get_description(): str
Rock
+ get_description(): str
NPC
- dialogue: string
+ __init__(name: str, dialogue: str)
+ speak()
Player
+ __init__(name: str)
EnvElementFactory
+ create_env_element(env_element_type: str): EnvElement | None
CharacterFactory
+ create_character(character_type: str, name: str, dialogue: str): Character | None

Code

```
"""
Joseph Leone
CEN 3062C
Assignment 3
Due Date: 8/2/2026
Professor: Dr. Kayikci
Description: This program creates a grid with environment elements and NPC
characters.
The user can give commands to move around the grid to discover the
entities in the world.
There is an abstract factory class to handle the creation of two types of
factories.
One of those factories is for making Character objects and the other is
for making
EnvironmentElement objects. There is a World class that is a composition
of these elements
and it manages their locations.
"""
from abc import ABC, abstractmethod

def main():
    # Create the world, characters, and environment elements
    world = World()
    characterFactory = MainFactory.get_factory("character")
    envFactory = MainFactory.get_factory("environment")
    player = characterFactory.create_character("player", name="Joseph")
    npc1 = characterFactory.create_character("npc", name="Mike",
dialogue="Are you lost?")
    npc2 = characterFactory.create_character("npc", name="Jasmine",
dialogue="Hi, I'm Jasmine. I'm glad to see you")
    tree1 = envFactory.create_env_element("tree")
    tree2 = envFactory.create_env_element("tree")
    rock1 = envFactory.create_env_element("rock")

    # Place the entities in the world
    world.place(npc1, 2, 2)
    world.place(npc2, 9, 9)
    world.place(tree1, 9, 8)
```

```
    world.place(tree2, 7, 9)
    world.place(rock1, 7, 8)
    world.place(player, 4, 4)

    # Welcome the player and show controls
    print("Welcome!")
    print("Controls: w = up, s = down, a = left, d = right, x = exit")

    # Game loop
    while(True):
        # Show the user their current poision and prompt them to enter a
command
        print(f"\nYou're at ({player.get_x()}, {player.get_y()})")
        move = input("Enter w, s, a, or d to move or x to exit: ").strip()

        # Process the user's input and respond
        if move == "w":
            move_x = player.get_x()
            move_y = player.get_y() + 1
            message = "You moved up"
        elif move == "s":
            move_x = player.get_x()
            move_y = player.get_y() - 1
            message = "You moved down"
        elif move == "a":
            move_x = player.get_x() - 1
            move_y = player.get_y()
            message = "You moved left"
        elif move == "d":
            move_x = player.get_x() + 1
            move_y = player.get_y()
            message = "You moved right"
        elif move == "x":
            print("Thanks for playing")
            break
        else:
            print("Invalid instruction")
            continue
```

```
        # Check to make sure if the user moves that they will still be in
the world's grid
        if world.is_in_range(move_x, move_y) == False:
            print("Out of range of the world, turn back.")
            continue

        # Attempt to get the entity at the coordinates the player wants to
move to.
        # If no entity is there move the user. If an entity is there show
don't allow
        # the user to move and give feedback from the entity they
encountered.
        entity_at_move_coords = world.get_entity_at(move_x, move_y)
        if entity_at_move_coords == None:
            world.move(player, move_x, move_y)
            print(message)
        else:
            if hasattr(entity_at_move_coords, "get_description"):
                print(f"You encountered
{entity_at_move_coords.get_description()}")
            elif hasattr(entity_at_move_coords, "get_name"):
                print(f"You encountered
{entity_at_move_coords.get_name()}")
                if hasattr(entity_at_move_coords, "speak"):
                    entity_at_move_coords.speak()
            else:
                print("Unable to move that way")


class Entity:
    """ Parent class for Character and EnvironmentElement containing
coordinate attributes and getter and setter methods for them
    attributes:
        int x: the position across the horizontal axis (column)
        int y: the position across the vertical axis (row)
    """
    def __init__(self):
        self._y = None
        self._x = None
```

```
    def set_y(self, y):
        self._y = y

    def set_x(self, x):
        self._x = x

    def get_y(self):
        return self._y

    def get_x(self):
        return self._x


class Character(Entity):
    """Inherits from Entity and adds a name attribute and a getter
function for it
    attributes:
        string name: name of the character
    """
    def __init__(self, name):
        super().__init__()
        self._name = name

    def get_name(self):
        return self._name


class NPC(Character):
    """Inherits from Character and adds a dialogue attribute and a speak
method to print the character's name and dialogue
    attributes:
        string dialogue: the string that is outputed when the character is
encourtered in the world
    """
    def __init__(self, name, dialogue):
        super().__init__(name)
        self._dialogue = dialogue

    def speak(self):
        print(f"{self.get_name()}: {self._dialogue}")
```

```
class Player(Character):
    """Inherits from Character. The class for the moveable character"""
    def __init__(self, name):
        super().__init__(name)


class CharacterFactory:
    """Factory to create the two different character types: player or
npc"""
    @staticmethod
    def create_character(character_type, **kwargs):
        """Static method that takes in a string to identify which
Character object is to be created. NPC characters have dialogue which
needs to be added as a keyword argument.
        inputs:
            string character_type: used to identify which type of
Character object is to be created
            **kwargs: used to add dialogue if creating an NPC Character
        outputs:
            NPC: if character_type = "npc"
            Player: if character_type = "player"
            None: if the character_type string doesn't match
            """
        if character_type == "npc":
            return NPC(kwargs["name"], kwargs["dialogue"])
        elif character_type == "player":
            return Player(kwargs["name"])
        else:
            return None


class EnvElement(Entity, ABC):
    """Inherits from Entity. Has an abstract method that is implemented in
child classes"""
    def __init__(self):
        super().__init__()

    @abstractmethod
```

```
    def get_description(self):
        """Returns the description of the EnvElement object"""
        pass


class Tree(EnvElement):
    """Inherits from EnvElement. Implements abstract method from parent
get_description. Tree element to be placed in the world"""
    def get_description(self):
        return "a strong oak tree with green leaves"


class Rock(EnvElement):
    """Inherits from EnvElement. Implements abstract method from parent
get_description. Rock element to be placed in the world"""
    def get_description(self):
        return "a large boulder than can't be moved"


class EnvElementFactory:
    """Factory to create Rock and Tree instances"""
    @staticmethod
    def create_env_element(env_element_type):
        if env_element_type == "tree":
            return Tree()
        elif env_element_type == "rock":
            return Rock()
        else:
            return None


class MainFactory:
    """Abstract factory with a method, get_factory to get either the
CharacterFactory or EnvElementFactory"""
    @staticmethod
    def get_factory(factory_type):
        if factory_type == "character":
            return CharacterFactory()
        elif factory_type == "environment":
            return EnvElementFactory()
```

```
        else:
            return None

class World:
    """This class create a grid and is used to manage the location and
movement of entities on it.
    It uses the singleton pattern so there can only be one instance of
World in the program
    """
    _instance = None
    ROWS = 10
    COLUMNS = 10

    def __new__(cls):
        """Handle singleton pattern and initializes the grid with all None
values"""
        if not cls._instance:
            cls._instance = super().__new__(cls)
            cls._instance._locations = [[None for _ in range(cls.COLUMNS)]
for _ in range(cls.ROWS)]
        return cls._instance

    def _is_valid_position(self, x, y):
        """Checks if the coordinates are in range of the grid and if there
is no entity occupying that position already"""
        if self.is_in_range(x, y) == False:
            print("out of range of the world")
            return False
        elif self._locations[y][x] != None:
            print("something here already")
            return False
        else:
            return True

    def is_in_range(self, x, y):
        """Checks to see if the coordinates are in range of the grid"""
        if x not in range(0, self.COLUMNS) or y not in range(0,
self.ROWS):
            return False
        else:
```

```
            return True

    def place(self, entity, x, y):
        """Places the entity at the coordinates in the world"""
        if self._is_valid_position(x, y):
            self._locations[y][x] = entity
            entity.set_y(y)
            entity.set_x(x)

    def move(self, entity, new_x, new_y):
        """Moves an entity from its current coordinates to new_x, new_y in
the world"""
        self._locations[entity.get_y()][entity.get_x()] = None
        self._locations[new_y][new_x] = entity
        entity.set_y(new_y)
        entity.set_x(new_x)

    def get_entity_at(self, x, y):
        """Returns a reference for the entity instance at x, y"""
        return self._locations[y][x]


if __name__ == "__main__":
    main()
```